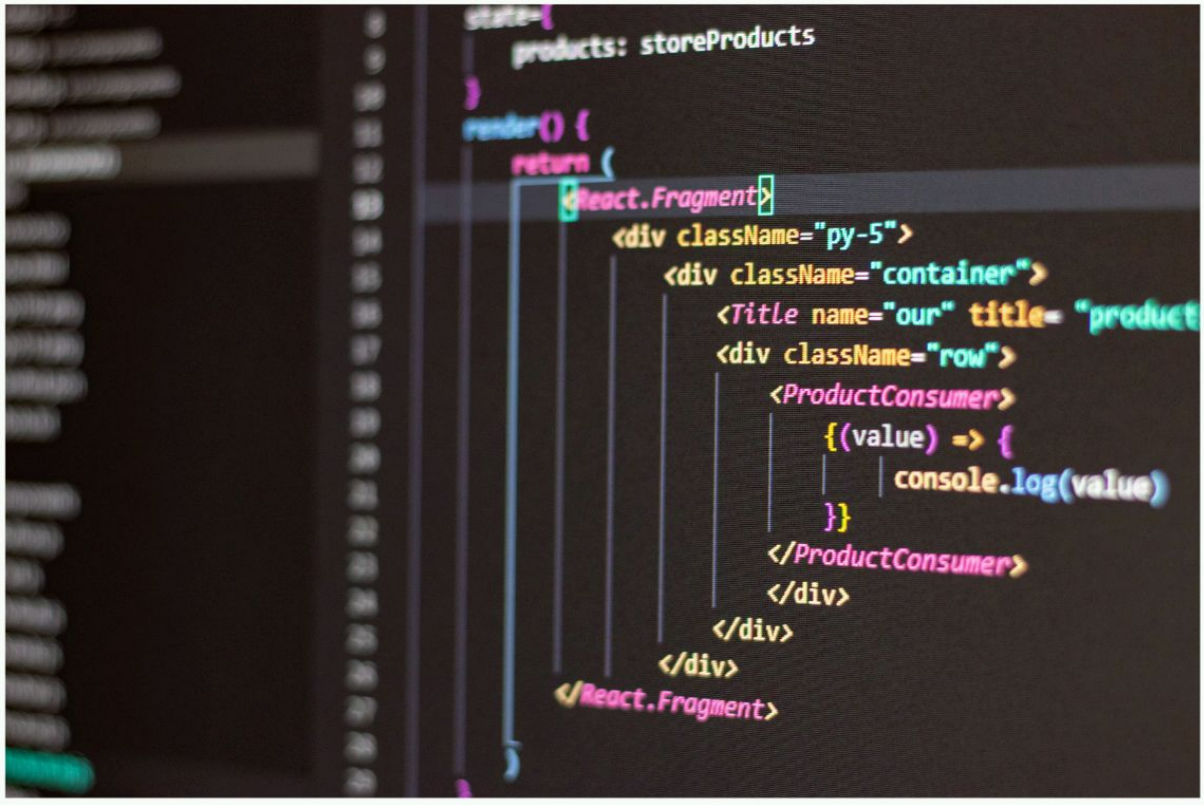
ARDUNOVA

MANUAL DE CÓDIGO C++

Fundamentos de programación para proyectos con Arduino

APRENDER · EXPERIMENTAR · CREAR

Guías de consulta para estudiantes y personas que comienzan en electrónica y programación.



1. ¿Qué es C++?

C++esun lenguaje de programación utilizado para desarrollar distintos tipos de programas y sistemas. El entorno de Arduino utiliza una forma de programación basada en C/C++, junto con funciones y bibliotecas que simplifican el control de la placa y sus componentes.

Programar consiste en escribir instrucciones ordenadas para que el dispositivo realice una tarea. Para que el programa funcione, las instrucciones deben respetarlas reglas de escritura del lenguaje.

2. Estructura de un programa de Arduino

```
void setup(){

 // Se ejecuta al comenzar
}
void loop() {
 // Se repite continuamente
}
```

void indica que la función no devuelve un valor. **setup()** configura el inicio del programa y **loop()** contiene las accionesquese repiten. Las llaves delimitanel bloque deinstrucciones.

3. Reglas básicas de escritura

- Las instrucciones suelen terminarcon punto y coma(;).
- Las llaves { } agrupan instrucciones.
- Los paréntesis ( ) se usan en funciones y condiciones.
- C++ distingue entre mayúsculas y minúsculas: **valor** y **Valor** son nombres distintos.
- Los comentarios ayudan a explicar el código y no se ejecutan.

Ejemplo de comentario: // Enciende el LED

4. Variables y tipos de datos

Una variable guarda un dato que el programa puede consultar o modificar. El tipo indica qué clase de dato almacena.

int: números enteros.
float: números con parte decimal.
char: un carácter.
bool: verdadero o falso (true/false).
String: texto en entornos Arduino que admiten este tipo.

Ejemplo:

```
int led = 13;
float temperatura = 24.5;
bool encendido = true;
```

En este ejemplo, **led** guarda el número 13, **temperatura** contiene un valor decimal y **encendido** almacena un estado lógico.

5. Operadores

Los operadores permiten realizarcálculos y comparar valores.

Aritméticos: + suma, - resta, * multiplicación, / división, % resto de una división entera.
Comparación: == igual, != distinto, > mayor, < menor, >= mayor o igual, <= menor o igual.
Lógicos: && AND, || OR, ! NOT.

Importante: **=** asigna un valor,mientrasque**==**comparadosvalores.

6. Condicionales: if y else

Las condicionales permiten ejecutar instrucciones según secumpla o no una condición.

```
int luz = 1;

if (luz == 1) {
 // La condición es verdadera
} else {
 // La condición es falsa
}
```

En este ejemplo, el programa comprueba si la variable luz vale 1. Si la condición se cumple, ejecuta el primer bloque; de lo contrario, ejecuta el bloque de else.

7. Repeticiones: for y while

Los bucles permiten repetir instrucciones.**for** suele utilizarse cuando se conoce la estructura de repetición que se necesita; **while** repite las instrucciones mientras una condición sea verdadera.

```
for (int i = 0; i < 5; i++) {
 // Se repite cinco veces
}

int contador = 0;
while (contador < 3) {
 contador++;
}
```

En el bucle for, la variable i comienza en 0 y aumenta hasta que deja de cumplirse la condición. En el ejemplo con while, contador aumenta en cadavuelta hasta alcanzar3.

8. Funciones habituales de Arduino

pinMode(pin,modo): configura un pin como entrada o salida.
digitalWrite(pin, valor): establece un estado digital de salida.
digitalRead(pin): lee un estado digital.
analogRead(pin): lee una entrada analógica en las placas que la admiten.
analogWrite(pin, valor): genera una salida PWM en los pines compatibles; no significa que todas las placas produzcan una salida analógica real.
delay(ms): pausa el programa durante el tiempo indicado en milisegundos.

9. Ejemplo completo: parpadeo de LED

```
void setup() {
 pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
 digitalWrite(LED_BUILTIN, HIGH);
 delay(1000);
 digitalWrite(LED_BUILTIN, LOW);
 delay(1000);
}
```

El programa configura el LED integrado como salida. Después lo enciende, espera un segundo, lo apaga y vuelve a esperar.Como las instrucciones están dentro deloop(),la secuencias serepite continuamente.

10. Errores comunes y cómo corregirlos

Faltade puntoy coma: revisá el final de cada instrucción.
Llaves sin cerrar: comprobá que cada { tenga su correspondiente }.
Nombre mal escrito: verificá mayúsculas y minúsculas.
Variable no declarada: asegurate de definirla antes de usarla.
Condición incorrecta: distinguí entre asignación(=) y comparación(==).

11. Método para aprender a programar

Leé el código completo antes de ejecutarlo. Después, identificá qué hace cada parte y probá modificar un solo valor. Compilá el programa, observá el resultado y anotá los cambios. Si aparece un error, usá el mensaje del entorno como guía para localizarlo.

Conclusión

Comprender las bases de C++ ayuda a interpretar los programas de Arduino y crear proyectos propios. Las variables, condiciones, bucles y funciones son herramientas fundamentales para controlar circuitos y resolver problemas mediante la programación.

Referencia oficial: https://docs.arduino.cc/ y https://www.arduino.cc/en/software